Programação Orientada a Objetos em Java: Tratamento de Exceções

Dani Juliano Czelusniak, Prof. Dr.

Atualizado em 2026

1 Introdução ao Tratamento de Exceções

Na Programação Orientada a Objetos (POO), a construção de softwares robustos e confiáveis é uma premissa fundamental. No entanto, durante a execução de um programa, situações anômalas ou imprevistas podem ocorrer: um arquivo que o sistema tenta ler foi apagado, a rede cai no meio de uma transação financeira, ou um usuário insere um texto onde se esperava um número.

Em Java, essas situações excepcionais são tratadas por meio de um mecanismo nativo chamado **Tratamento de Exceções** (*Exception Handling*). Uma exceção é, na verdade, um objeto que sinaliza que algo fora do fluxo normal do programa aconteceu. Quando um erro ocorre, o método atual "lança"(*throws*) um objeto de exceção. Se essa exceção não for "capturada"(*caught*) e tratada, o programa será encerrado de forma abrupta.

O grande objetivo do tratamento de exceções é separar o código que implementa a **lógica de negócio principal** do código que **lida com os erros**. Isso torna os sistemas mais limpos, legíveis e, acima de tudo, tolerantes a falhas.

1.1 A Hierarquia de Exceções em Java

Como tudo em Java se baseia em objetos, as exceções não são diferentes. Todas as exceções e erros derivam de uma superclasse raiz chamada `Throwable`.

- **Error:** Representa falhas graves no ambiente de execução (JVM), como falta de memória (`OutOfMemoryError`). O desenvolvedor não deve tentar capturar e tratar esses erros, pois a aplicação não tem como se recuperar deles.
- **Exception:** Representa problemas que um programa razoável deveria tentar capturar e se recuperar. Subdivide-se em duas categorias cruciais:
  - **Exceções Verificadas (*Checked Exceptions*):** O compilador obriga o programador a tratá-las (ex: `IOException`, `SQLException`). Se não forem tratadas, o código sequer compila.
  - **Exceções Não-Verificadas (*Unchecked Exceptions*):** Herdam de `RuntimeException`. Ocorrem devido a falhas de programação, como acessar posições inválidas de vetores ou divisão por zero (ex: `NullPointerException`, `ArithmeticException`). O compilador não obriga o seu tratamento, mas uma boa lógica deve preveni-las.

2 Estrutura Básica: try, catch e finally

Para proteger um trecho de código que é suscetível a falhas, utilizamos os blocos de captura da linguagem Java. Esta estrutura impede que a aplicação feche abruptamente e permite que o programador forneça uma resposta amigável ao usuário.

2.1 Diagrama e Componentes

O tratamento se baseia em três palavras-chave principais:

- **`try`:** Delimita o bloco de código "arriscado". Se ocorrer uma exceção aqui, o fluxo normal é interrompido imediatamente e o controle passa para o bloco `catch`.
- **`catch`:** Bloco de recuperação. Só é executado se uma exceção do tipo especificado ocorrer dentro do `try`. Pode haver múltiplos `catch` para um único `try`.
- **`finally`:** Bloco opcional que **sempre** será executado, ocorra ou não uma exceção. É utilizado principalmente para liberar recursos críticos (como fechar conexões de banco de dados ou fechar arquivos abertos).

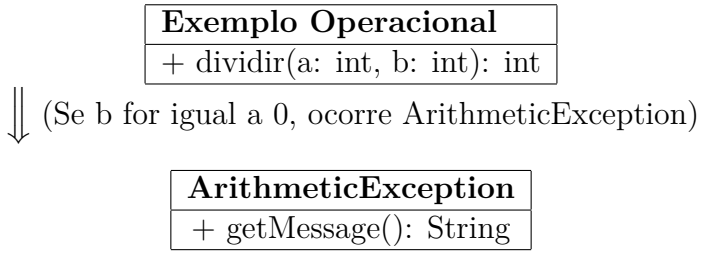


Figura 1: Diagrama conceitual do disparo de uma exceção não-verificada em uma operação matemática.

2.2 Implementação em Java

No exemplo abaixo, construímos uma classe matemática genérica e protegemos a chamada do método com um bloco `try-catch-finally`.

```
class Calculadora {
    public int dividir(int a, int b) {
        // Se 'b' for 0, a JVM criara internamente um objeto
   ArithmeticException e o lancara.
        return a / b;
    }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Inicio do Programa ---");
        Calculadora calc = new Calculadora();

        try {
            // Tentativa de executar codigo critico
            System.out.println("Tentando realizar divisao...");
```

```
16              int resultado = calc.dividir(10, 0); // Linha onde a
    excecao ocorre
17              System.out.println("Resultado: " + resultado); // Esta
    linha nao sera executada
18
19          } catch (ArithmeticException e) {
20              // Capturando o objeto da excecao e tratando
21              System.err.println("Erro capturado: Nao e possivel
    dividir por zero!");
22              // e.getMessage() traz detalhes tecnicos do erro
23              System.err.println("Detalhe tecnico: " + e.getMessage());
24
25          } finally {
26              // Este bloco SEMPRE e executado
27              System.out.println("Bloco finally: Limpando memoria,
    encerrando recursos...");
28          }
29
30          // O programa continua fluindo normalmente, evitando o
    encerramento abrupto (Crash)
31          System.out.println("--- Fim do Programa ---");
32      }
33  }
```

Listing 1: Utilizando blocos try, catch e finally

3 Lançando e Propagando: throw e throws

Além de tratar exceções que a própria linguagem gera (como falhas matemáticas), nós mesmos podemos e devemos impor as regras de negócio de nossos sistemas disparando erros propositais quando algo inválido for solicitado.

3.1 Diferença entre throw e throws

- **`throw`:** É a **ação** imediata de instanciar e arremessar uma exceção no momento em que a regra é violada. Ocorre **dentro** do corpo do método.
- **`throws`:** É o **aviso** na assinatura do método de que aquele trecho de código "é perigoso"e pode devolver uma exceção. Ele avisa quem for chamar o método que essa responsabilidade deverá ser tratada.

3.2 Implementação em Java

Neste exemplo, temos um sistema de senhas. Se a senha for inválida, disparamos a exceção ativamente com `throw`.

```
class ValidadorSeguranca {
    // A palavra "throws" na assinatura indica a propagacao do erro.
    // Qualquer classe que chamar validarSenha() TERA que lidar com um Exception.
    public void validarSenha(String senha) throws Exception {
        if (senha == null || senha.length() < 6) {
            // "throw" instancia o objeto do erro e encerra o fluxo do metodo imediatamente
            throw new Exception("A senha deve ter no minimo 6 caracteres.");
        }
        System.out.println("Senha aprovada pelos criterios de seguranca.");
    }
}

public class Main {
    public static void main(String[] args) {
        ValidadorSeguranca validador = new ValidadorSeguranca();

        try {
            // Como validarSenha tem "throws Exception", o compilador nos OBRIGA
            // a colocar este codigo dentro de um try-catch.
            validador.validarSenha("123");
        } catch (Exception e) {
            System.out.println("Alerta de Seguranca: " + e.getMessage());
        }
    }
}
```

Listing 2: Lançando exceções ativamente com throw

4 Criando Exceções Personalizadas

Tratar a classe genérica `Exception` funciona, mas em sistemas maduros orientados a objetos, precisamos de precisão. O ideal é representar os erros do nosso **Domínio de Negócio** como classes específicas.

Por exemplo, um erro de banco não é um `Exception` genérico, é um `SaldoInsuficienteException`. Criar uma exceção personalizada significa apenas criar uma classe que herda de `Exception` (para obrigar verificação) ou `RuntimeException` (para exceções opcionais).

4.1 Diagrama de Classes

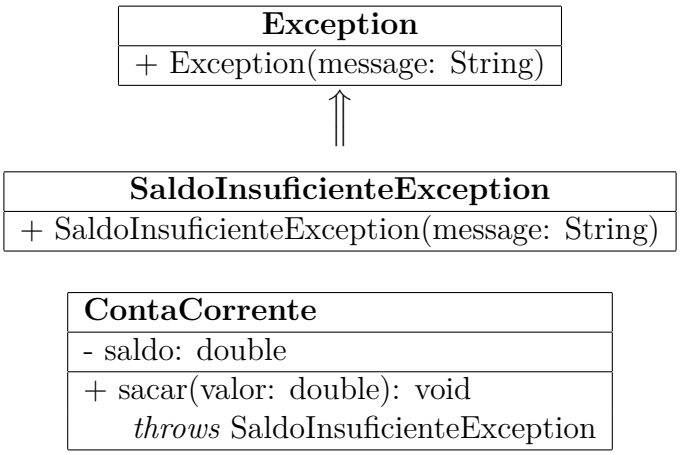


Figura 2: Diagrama de classes ilustrando herança na criação de Exceções Customizadas.

4.2 Implementação em Java

Vamos transformar o diagrama acima em código real, mostrando o nível máximo de controle e semântica na Orientação a Objetos.

```
// 1. Criacao da classe que representa nosso erro especifico de negocio
class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(String mensagem) {
        // Envia a mensagem para a superclasse Exception gerenciar
        super(mensagem);
    }
}

// 2. Classe de dominio que implementa a regra
class ContaCorrente {
    private double saldo = 100.0;

    public void sacar(double valor) throws SaldoInsuficienteException {
        if (valor > this.saldo) {
            // Ao inves de retornar um booleano "false", lancamos um Erro rico em semantica
            throw new SaldoInsuficienteException("Tentativa de saque de R$ " + valor +
```

```
                " negada. Saldo disponivel: R$ " + this.saldo);
        }
        this.saldo -= valor;
        System.out.println("Saque de R$ " + valor + " realizado com
    sucesso.");
     }
}

// 3. Teste no Main
public class Main {
     public static void main(String[] args) {
        ContaCorrente conta = new ContaCorrente();

        try {
            conta.sacar(50.0);  // Fluxo normal - ok
            conta.sacar(200.0); // Isso ira gerar a excecao
    personalizada
        } catch (SaldoInsuficienteException e) {
            // Tratamento ultra-especifico para erros de Saldo
            System.out.println("TRANSAÇÃO BLOQUEADA: " + e.getMessage
    ());
        }
     }
}
```

Listing 3: Criando e utilizando Exceções Personalizadas (Domínio)

5 Lista de Exercícios Práticos

Abaixo são apresentados 6 exercícios voltados a praticar não só o manuseio técnico de blocos `try-catch`, mas o raciocínio arquitetural na resolução de falhas.

5.1 Nível Fácil

Exercício 1: Proteção contra formato incorreto
Enunciado: Crie uma classe `Conversor` com um método estático que recebe uma `String` e a converte para `Integer` usando `Integer.parseInt()`. No método `main`, passe um texto não-numérico (ex: "ABC") e utilize `try-catch` para capturar o `NumberFormatException`, exibindo uma mensagem amigável.

```
class Conversor {
    public static int converterParaInt(String texto) {
        return Integer.parseInt(texto);
    }
}

public class Main {
    public static void main(String[] args) {
        String entradaInvalida = "ABC";

        try {
            int numero = Conversor.converterParaInt(entradaInvalida);
            System.out.println("O numero e: " + numero);
        } catch (NumberFormatException e) {
            System.out.println("Erro: Voce digitou letras ao inves de
    numeros.");
            System.out.println("Dado que tentou ser convertido: " +
   entradaInvalida);
        }
    }
}
```

Listing 4: Resolução: Exercício 1 - Fácil

Exercício 2: Tratamento de Ponteiro Nulo (Null Pointer)
Enunciado: Um dos erros mais clássicos do Java ocorre quando tentamos invocar um método sobre uma variável vazia. Crie uma classe com um objeto `String` recebendo `null`. Tente chamar o método `.length()` nela. Envolva em um bloco para tratar `NullPointerException`.

```
public class Main {
    public static void main(String[] args) {
        String textoVazio = null;

        try {
            // Tenta chamar metodo length() em um objeto que nao
   existe na memoria
            int tamanho = textoVazio.length();
            System.out.println("Tamanho: " + tamanho);
```

```
9           } catch (NullPointerException e) {
10              System.out.println("Falha grave capturada: Tentativa de
    acessar um objeto nulo!");
11              System.out.println("Certifique-se de que a variavel foi
    inicializada (instanciada).");
12          }
13      }
14  }
```

Listing 5: Resolução: Exercício 2 - Fácil

5.2 Nível Médio

Exercício 3: Múltiplas Exceções simultâneas (Multi-catch)
Enunciado: Crie um vetor de inteiros com 3 posições. Tente realizar duas operações perigosas seguidas no mesmo bloco: (1) acessar o índice 5 do vetor e (2) fazer uma divisão por zero. Crie um `catch` específico para índice inválido (`ArrayIndexOutOfBoundsException`) e outro para erro matemático (`ArithmeticException`). Analise qual deles é executado primeiro.

```
public class Main {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30};

        try {
            // Acao perigosa 1: Acessar posicao alem do limite
            System.out.println("Valor na posicao 5: " + numeros[5]);

            // Acao perigosa 2: Divisao por zero
            // Nota: Esta acao sequer sera alcancada, pois a linha superior interrompe o fluxo.
            int resultado = 100 / 0;

        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("ERRO DE VETOR: Voce tentou acessar uma posicao que nao existe.");
        } catch (ArithmeticException e) {
            System.out.println("ERRO MATEMATICO: Impossivel realizar divisao por zero.");
        } catch (Exception e) {
            // Boa pratica: um catch generico no final para imprevistos
            System.out.println("Um erro desconhecido ocorreu: " + e.getMessage());
        }
    }
}
```

Listing 6: Resolução: Exercício 3 - Médio

Exercício 4: Arquitetura Simples de Banco de Dados com Finally
Enunciado: Crie a classe `BancoDeDadosMock` com o método `conectar()` (que imprime "Conexão Aberta") e `desconectar()` ("Conexão Fechada"). No `main`, abra a conexão. Depois, lance forçadamente um erro `RuntimeException` usando `throw`. Utilize a cláusula `finally` para garantir que o banco feche, independentemente do erro.

```
class BancoDeDadosMock {
    public void conectar() { System.out.println("[DB] Conexao Aberta."); }
    public void desconectar() { System.out.println("[DB] Conexao Fechada com Seguranca."); }
}

```

```java
public class Main {
    public static void main(String[] args) {
        BancoDeDadosMock db = new BancoDeDadosMock();

        try {
            db.conectar();
            System.out.println("Tentando inserir dados...");
            // Simulando um erro grave durante o insert
            throw new RuntimeException("Tabela 'usuarios' corrompida!");

        } catch (RuntimeException e) {
            System.out.println("FALHA CAPTURADA: " + e.getMessage());

        } finally {
            // Mesmo com a falha acima, o finally garante a sanidade dos recursos
            System.out.println("Iniciando rotina de limpeza do Finally...");
            db.desconectar();
        }
    }
}
```

Listing 7: Resolução: Exercício 4 - Médio

5.3 Nível Difícil

Exercício 5: Validador de Negócio (Exception Customizada + Propagação)
Enunciado: Crie uma `IdadeInvalidaException` que herde de `Exception`. Crie uma classe `SistemaCNH` com o método `processarCadastro(int idade)`. O método deve lançar a sua exceção customizada se a idade for menor que 18, propagando via `throws`. No `main`, aplique um laco de repetição em um vetor de idades `{15, 20, 17, 30}` tentando cadastrar a todos, parando a operação inteira (abortando o cadastro em lote) se a exceção acontecer, mas informando no console quem causou o erro.

```
// Excecao customizada e CHECKED (Exige try-catch)
class IdadeInvalidaException extends Exception {
    public IdadeInvalidaException(String msg) { super(msg); }
}

class SistemaCNH {
    public void processarCadastro(int idade) throws
   IdadeInvalidaException {
        if (idade < 18) {
            throw new IdadeInvalidaException("Cadastro bloqueado.
   Idade recebida: " + idade + " anos.");
        }
        System.out.println("CNH processada com sucesso para candidato
    de " + idade + " anos.");
    }
}

public class Main {
    public static void main(String[] args) {
        SistemaCNH sistema = new SistemaCNH();
        int[] candidatos = {20, 30, 15, 25}; // O candidato de 15
   abortara o resto

        try {
            for (int i = 0; i < candidatos.length; i++) {
                System.out.print("Avaliando candidato " + (i+1) + "
   -> ");
                sistema.processarCadastro(candidatos[i]);
            }
            System.out.println("Todos os cadastros foram processados
   com sucesso.");
        } catch (IdadeInvalidaException e) {
            // No momento do erro (candidato 3), o laco for e
   interrompido.
            System.out.println("\nFALHA NO LOTE: " + e.getMessage());
            System.out.println("Processo abortado preventivamente.");
        }
    }
}
```

Listing 8: Resolução: Exercício 5 - Difícil

Exercício 6: Integração Polimorfismo e Tratamento em Lote Resiliente
Enunciado: Aqui unimos todos os conceitos. Crie a exceção `EstoqueInsuficienteException`. Crie a classe `Produto` com um `nome` e um `estoque`. Crie um método `vender(int quantidade) throws EstoqueInsuficienteException` que debita do estoque ou lança a falha. No método `main`, crie um vetor com 3 produtos e efetue um pedido de compra (ex: 5 unidades) para TODOS. **O grande desafio:** Diferente do exercício anterior, se um produto não tiver estoque, capture a exceção e registre o erro, mas **continue** o `for` para vender os demais. O sistema de lote deve ser resiliente.

```
class EstoqueInsuficienteException extends Exception {
    public EstoqueInsuficienteException(String msg) { super(msg); }
}

class Produto {
    private String nome;
    private int quantidadeEmEstoque;

    public Produto(String nome, int quantidadeInicial) {
        this.nome = nome;
        this.quantidadeEmEstoque = quantidadeInicial;
    }

    public void vender(int qtdSolicitada) throws
   EstoqueInsuficienteException {
        if (qtdSolicitada > this.quantidadeEmEstoque) {
            throw new EstoqueInsuficienteException("Faltam produtos
   para " + this.nome +
                ". Solicitado: " + qtdSolicitada + " | Disponivel: "
   + this.quantidadeEmEstoque);
        }
        this.quantidadeEmEstoque -= qtdSolicitada;
        System.out.println("Venda confirmada: " + qtdSolicitada + "x
   " + this.nome);
    }
}

public class Main {
    public static void main(String[] args) {
        Produto[] inventario = new Produto[3];
        inventario[0] = new Produto("Notebook", 10);
        inventario[1] = new Produto("Monitor", 2); // Estoque critico
        inventario[2] = new Produto("Teclado", 50);

        System.out.println("--- Inciando Processamento de Pedidos em
   Lote ---");

        for (int i = 0; i < inventario.length; i++) {
            // O try-catch DENTRO do laco de repeticao cria
   RESILIENCIA.
            // Se um erro ocorrer nesta interacao, ela e capturada,
   mas o 'for' continua.
            try {
```

```
37                  // Simulando que o cliente solicitou 5 unidades de
    cada item
38                  inventario[i].vender(5);
39              } catch (EstoqueInsuficienteException e) {
40                  System.out.println("[ALERTA DE ESTOQUE] " + e.
    getMessage());
41              }
42          }
43
44          System.out.println("--- Lote concluido! O sistema nao sofreu
    crash ---");
45      }
46  }
```

Listing 9: Resolução: Exercício 6 - Difícil